Parser 만들기
스펙표 설명 (1)

- Mini-Rust EBNF 문법 - 1

```
program     ::= decl* EOF ;

decl        ::= fnDecl ;

fnDecl      ::= 'fn' ID '(' params? ')' ( '->' type )? block ;

params      ::= param ( ',' param )* ;

param       ::= ID ':' type ;

// 타입은 i32, bool 두 가지
type        ::= 'i32' | 'bool' ;

block       ::= '{' stmt* '}' ;
```

Parser 만들기

스펙표 설명 (2)

- Mini-Rust EBNF 문법 - 2

```
// 타입은 i32, bool 두 가지
type        ::= 'i32' | 'bool' ;

block       ::= '{' stmt* '}' ;

stmt        ::= letStmt
              | ifStmt
              | whileStmt
              | returnStmt
              | printStmt
              | exprStmt ;
```

Parser 만들기
스펙표 설명 (3)

- Mini-Rust EBNF 문법 - 3

```
// 변수 선언: 초기값은 반드시 있어야 함, 타입 표기는 생략 가능
letStmt     ::= 'let' 'mut'? ID ( ':' type )? '=' expr ';' ;

// else 뒤에는 블록 또는 다시 if문 (else if 연결 가능)
ifStmt      ::= 'if' expr block ( 'else' ( block | ifStmt ) )? ;

whileStmt   ::= 'while' expr block ;

returnStmt  ::= 'return' expr? ';' ;

printStmt   ::= 'print' '(' expr ')' ';' ;

exprStmt    ::= expr ';' ;
```

Parser 만들기

스펙표 설명 (4)

- Mini-Rust EBNF 문법 - 4 (연산자 우선순위 주의)

```
expr               ::= assignExpr ;
assignExpr         ::= logicalOrExpr ( '=' assignExpr )? ;
logicalOrExpr      ::= logicalAndExpr ( '||' logicalAndExpr )* ;
logicalAndExpr     ::= equalityExpr ( '&&' equalityExpr )* ;
equalityExpr       ::= additiveExpr ( ( '==' | '!=' | '<' | '<=' | '>' | '>=' ) additiveExpr )* ;
additiveExpr       ::= multiplicativeExpr ( ( '+' | '-' ) multiplicativeExpr )* ;
multiplicativeExpr ::= unaryExpr ( ( '*' | '/' ) unaryExpr )* ;
```

Parser 만들기

스펙표 설명 (5)

- Mini-Rust EBNF 문법 - 5

```
// 음수 리터럴은 따로 없음: -1 은 단항 '-' 와 INT 1
unaryExpr          ::= '-' unaryExpr
                     | '!' unaryExpr
                     | primaryExpr ;

// ID 다음 토큰이 '(' 이면 함수 호출, 아니면 변수 참조
primaryExpr        ::= ID ( '(' args? ')' )?
                     | INT
                     | 'true'
                     | 'false'
                     | '(' expr ')' ;

args               ::= expr ( ',' expr )* ;
```